SIMATS ENGINEERING

ASSESSMENT TOOL 1: Experiments

COURSE NAME:	Cloud Computing and Big Data Analytics	COURSE CODE:	CSA15

COURSE OUTCOME COVERED

CO3: Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)

Assessment Tool Weightage: 40%

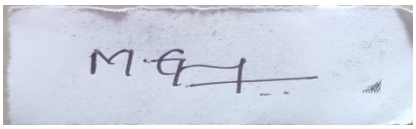
Dave's Taxonomy: Precision (Level 3) | Question Count: 5	Total Marks: 40

Code of Conduct

I Maddirala Gopi Krishna Reddy 192372055 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assessment Tasks

Experiments

1. Experiment 1: Create and document a cloud storage comparison between object, block, and file storage. Select the best storage model for backups, VM disks, and shared enterprise documents.
2. Experiment 2: Develop a simple REST API workflow for a cloud-based student portal. Show request, response, authentication, and deployment flow.
3. Experiment 3: Design a serverless event flow for image upload processing using object storage, function-as-a-service, and notification service.
4. Experiment 4: Compare edge computing and fog computing for a smart traffic management system. Include architecture, latency considerations, and deployment location.
5. Experiment 5: Demonstrate a cloud application deployment workflow showing client access, browser interaction, web API call, storage access, and monitoring.

For each experiment, include objective, architecture sketch, procedure, expected output, and conclusion.

Experiments Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Experimental Design	25%	Design is systematic and technically strong	Mostly correct design	Basic design with gaps	Poorly designed activity

Use of Tools / Platforms	20%	Appropriate and effective use of cloud tools	Good tool usage with minor issues	Limited tool usage	Incorrect or missing tool usage
Application of Concepts	20%	Concepts are applied accurately to practical tasks	Mostly accurate application	Partial application	Weak application
Analysis and Output	20%	Strong analysis and meaningful outcome interpretation	Good analysis with minor gaps	Basic analysis	Little or no analysis
Documentation	15%	Clear, structured, and complete documentation	Mostly complete documentation	Partially complete documentation	Poor documentation

Experiment 1: Cloud Storage Comparison — Object, Block, and File Storage

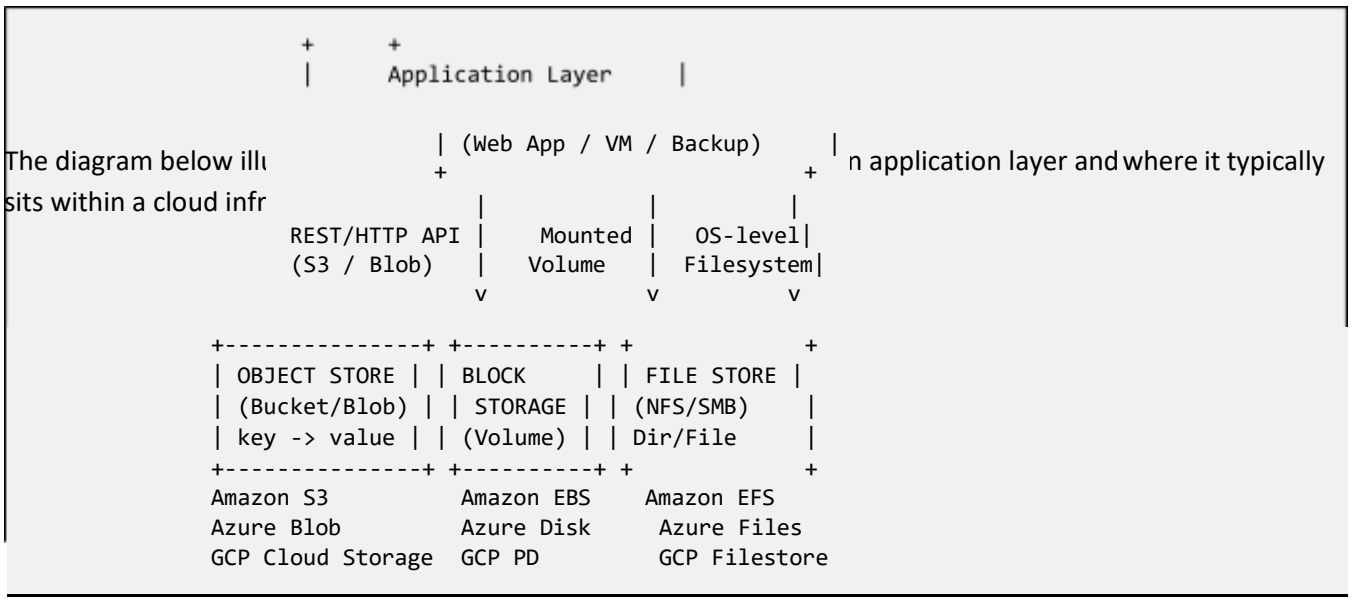
1. Objective

The objective of this experiment is to study and compare the three fundamental cloud storage models — Object Storage, Block Storage, and File Storage — in terms of their architecture, access methods, performance characteristics, scalability, and cost. Based on this comparison, the experiment further aims to identify and justify the most suitable storage model for three real-world use cases: long-term backups, virtual machine (VM) disks, and shared enterprise documents.

Understanding the trade-offs among these storage types is essential for any cloud architect, since choosing the wrong storage class can lead to unnecessary cost, poor application performance, or operational bottlenecks. This experiment therefore documents the decision-making process a cloud engineer would follow before provisioning storage resources on a public cloud platform such as AWS, Azure, or GCP.

2. Architecture Sketch

The diagram below illu n application layer and where it typically sits within a cloud infr



3. Procedure

1. Define the three storage categories conceptually: object (flat namespace, accessed via REST API, stores data as objects with metadata), block (raw storage volumes divided into fixed-size blocks, attached to a single compute instance), and file (hierarchical directory structure shared over a network protocol such as NFS/SMB).
2. Provision a sample bucket/container (e.g., Amazon S3 bucket) and upload a test file using the console or CLI (aws s3 cp test.txt s3://demo-bucket/) to observe object storage behaviour — note the returned object key and metadata.
3. Provision a block volume (e.g., Amazon EBS) and attach it to a running EC2 instance; format and mount it (mkfs, mount) to observe that it behaves like a local raw disk usable only by the attached VM.
4. Provision a managed file share (e.g., Amazon EFS) and mount it simultaneously on two different EC2

instances using NFS to observe concurrent multi-client access.

5. Benchmark basic read/write latency on each storage type using a simple tool (e.g., dd or fio) and record throughput and latency values.

6. Tabulate the comparison across durability, latency, scalability, access protocol, and typical pricing tier.
7. Map each storage type against the three target workloads — backups, VM disks, shared enterprise documents — and justify the best fit for each based on the tabulated criteria.

4. Expected Output

The experiment produces a comparative matrix similar to the one below, along with working demonstrations of upload/mount operations on each storage class.

Criteria	Object Storage	Block Storage	File Storage
Access Method	REST API (HTTP/HTTPS)	Raw device / mounted volume	Network protocol (NFS/SMB)
Data Unit	Object (data + metadata)	Fixed-size block	File within directory tree
Attach Scope	Any number of clients	Single instance (mostly)	Multiple clients concurrently
Scalability	Virtually unlimited	Limited by volume size	Elastic, grows automatically
Latency	Higher (ms range)	Very low (sub-ms)	Low-moderate
Best Fit	Backups, archives, static assets	VM boot/data disks, databases	Shared documents, home directories
Example Service	Amazon S3 / Azure Blob	Amazon EBS / Azure Disk	Amazon EFS / Azure Files

Use Case	Recommended Storage	Justification
Long-term Backups	Object Storage	Low cost per GB, high durability (11 nines), lifecycle policies for archival tiers
VM Disks	Block Storage	Low latency and high IOPS required for OS and application performance
Shared Enterprise Documents	File Storage	Native hierarchical structure and simultaneous multi-user (POSIX) access

5. Conclusion

This experiment demonstrated that no single cloud storage model is universally optimal; each is engineered for a distinct access pattern. Object storage excels at storing massive volumes of infrequently accessed, immutable data such as backups and archives because of its flat namespace, near-infinite scalability, and low per-GB cost. Block storage, in contrast, is optimised for low-latency, high-IOPS random access, making it the natural choice for VM boot disks and transactional databases where performance is critical. File storage bridges the two by offering a familiar hierarchical namespace with native support for concurrent multi-client access, which makes it ideal for shared enterprise documents and collaborative workloads.

The exercise reinforces those principle that storage selection in cloud architecture should be driven by the access pattern, durability requirement, latency sensitivity, and cost profile of the workload rather than by convenience

alone. Applying the wrong storage class — for example, using block storage for static backup archives — would result in unnecessary cost and operational complexity, whereas the recommended mapping (object → backups, block → VM disks, file → shared documents) yields the best balance of performance, scalability, and cost efficiency.

Experiment 2: REST API Workflow for a Cloud-Based Student Portal

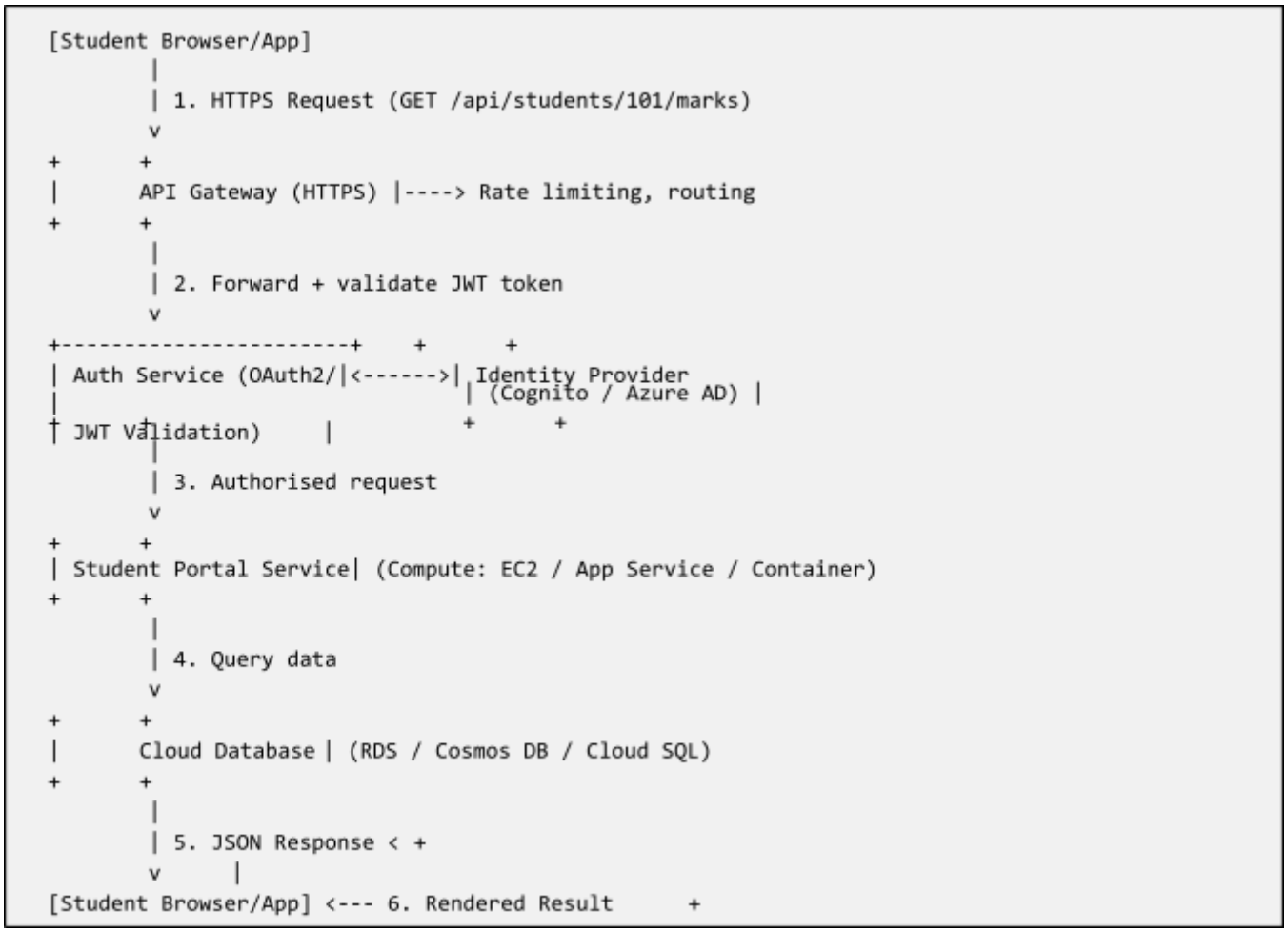
1. Objective

The objective of this experiment is to design and demonstrate a complete REST API workflow for a cloud-hosted student portal application. The experiment covers the full lifecycle of a client request — from the initial HTTP request, through authentication and authorisation, to the server response — and documents how such an API would be deployed on a cloud platform using managed compute and API gateway services.

The student portal exposes endpoints that allow students to view their profile, check attendance, and download marks. The experiment is designed to illustrate REST principles (statelessness, resource-based URIs, standard HTTP verbs) in a practical, deployable context.

2. Architecture Sketch

The following diagram shows the end-to-end flow of a client request through the deployed cloud architecture.



3. Procedure

8. Define the REST resources for the portal, e.g. /api/students/{id}, /api/students/{id}/attendance, /api/students/{id}/marks, using nouns rather than verbs in the URI path.
9. Design the request/response contract for each endpoint using standard HTTP methods: GET to retrieve, POST to create, PUT/PATCH to update, DELETE to remove.

10. Set up an identity provider (e.g., AWS Cognito or Azure AD B2C) to issue JWT access tokens on successful login of a student's credentials.
11. Deploy a lightweight backend service (e.g., Node.js/Express or Spring Boot) exposing the defined endpoints, and containerise it or deploy it to a managed compute service.
12. Place an API Gateway in front of the service to handle HTTPS termination, request throttling, and routing to the backend.
13. Configure the API Gateway (or the backend middleware) to validate the JWT bearer token on every incoming request before forwarding it to the service layer.
14. Test the flow end-to-end using a REST client (Postman/cURL): first call the login endpoint to obtain a token, then call a protected resource endpoint passing the token in the Authorization header.
15. Deploy the complete stack to the cloud environment and verify the same flow works over the public HTTPS endpoint.

4. Expected Output

A successful login request returns a signed JWT token as shown below:

```
POST /api/auth/login { "regNo": "CSA15021", "password": "********" } Response 200 OK {
"accessToken": "eyJhbGciOiJIUzI1NiIs...", "expiresIn": 3600 }
```

A subsequent authorised request to fetch marks returns:

```
GET /api/students/101/marks Authorization: Bearer eyJhbGciOiJIUzI1NiIs... Response 200 OK { "regNo":
"CSA15021", "subject": "CSA15", "marks": 92, "grade": "A+" }
```

An unauthorised request (missing/invalid token) is expected to return a 401 Unauthorized status, and a request for a resource owned by a different student should return 403 Forbidden, confirming that both authentication and authorisation are enforced correctly by the deployed workflow.

5. Conclusion

This experiment demonstrated the full request-response cycle of a RESTful cloud application, from client request through gateway routing, token-based authentication, business logic execution, and database interaction, back to a JSON response. The use of an API Gateway decouples the client from backend scaling concerns, while JWT-based authentication ensures the API remains stateless and horizontally scalable — a core tenet of REST architecture.

The deployment pattern demonstrated here — API Gateway, managed identity provider, containerised compute, and managed database — mirrors production-grade cloud-native application design and can be extended with features such as caching (API Gateway response caching), observability (centralized logging/metrics), and CI/CD pipelines for automated deployment of the student portal service.

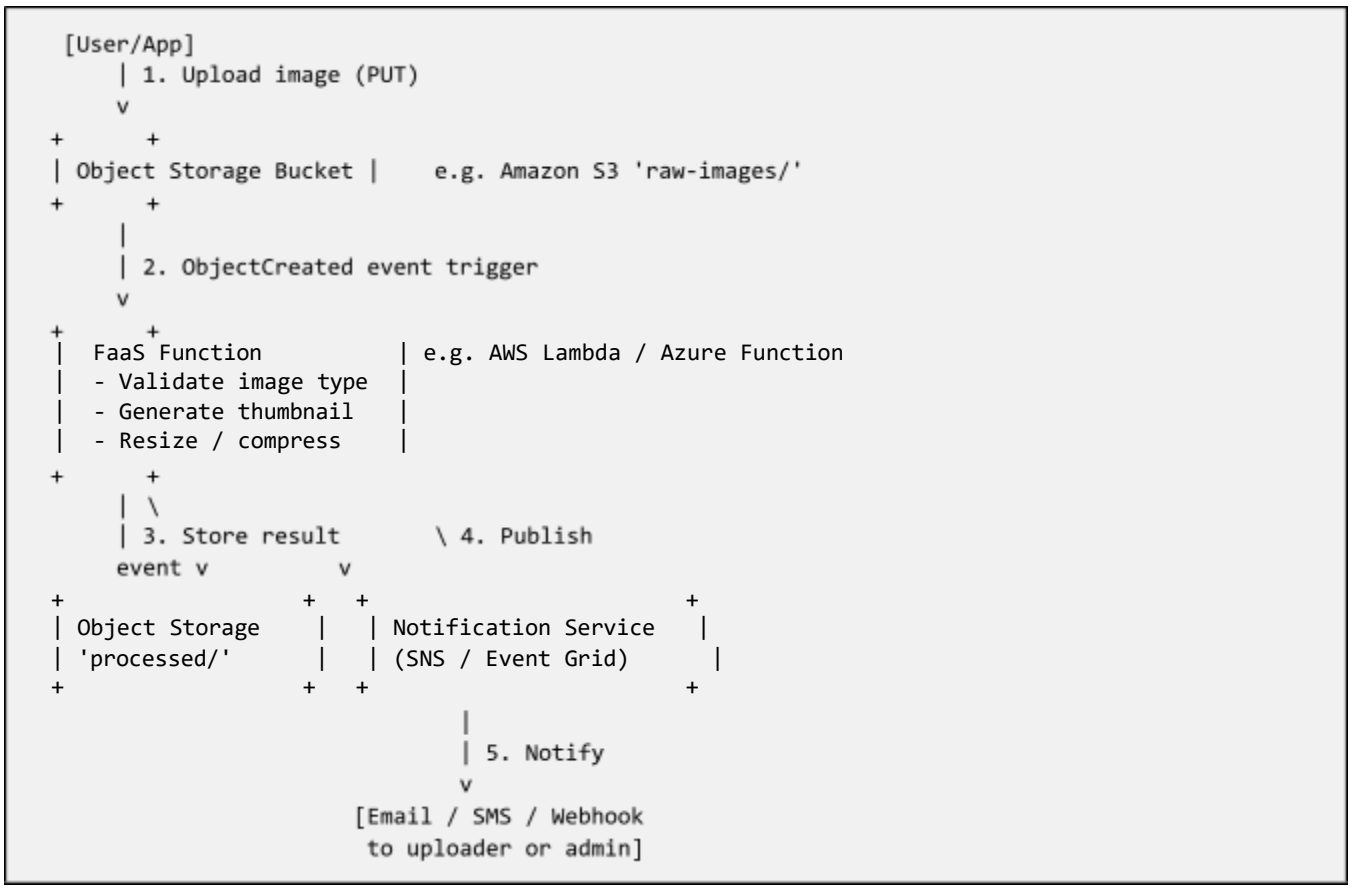
Experiment 3: Serverless Event Flow for Image Upload Processing

1. Objective

The objective of this experiment is to design a serverless event-driven architecture that automatically processes images as soon as they are uploaded to cloud object storage. The pipeline uses a Function-as-a-Service (FaaS) component to perform processing (e.g., thumbnail generation or format validation) and a notification service to alert relevant stakeholders once processing completes, without provisioning or managing any dedicated servers.

This experiment highlights the event-driven paradigm central to serverless computing, where cloud resources are invoked only in response to specific triggers, leading to cost efficiency (pay-per-execution) and automatic scaling.

2. Architecture Sketch



3. Procedure

16. Create an object storage bucket with two logical prefixes/folders: raw-images/ for incoming uploads and processed/ for output thumbnails.
17. Write a FaaS function (e.g., AWS Lambda in Python/Node.js) that accepts an S3 event payload, downloads the uploaded object, validates that it is a supported image type (JPEG/PNG), and generates a resized thumbnail using an imaging library.
18. Configure an event trigger on the bucket so that every ObjectCreated event under raw-images/ automatically invokes the FaaS function (S3 Event Notification -> Lambda trigger).
19. Within the function, upload the generated thumbnail back to the processed/ prefix of the same or a separate bucket.

20. Configure the function to publish a completion message to a notification/messaging service (e.g., Amazon SNS topic or Azure Event Grid) once processing succeeds.
21. Subscribe an email address or webhook endpoint to the notification topic to receive real-time alerts.
22. Test the pipeline by uploading a sample image to raw-images/ and observing (a) the FaaS execution logs, (b) the generated thumbnail in processed/, and (c) the notification received.
23. Introduce an invalid file (e.g., a .txt renamed as .jpg) to verify the function's validation logic correctly rejects it and raises an error notification instead.

4. Expected Output

Cloud function execution logs are expected to resemble the following:

```
START RequestId: 3f2a-91cd Event: raw-images/photo1.jpg (245 KB) Validating image type... OK
(image/jpeg) Generating thumbnail (200x200)... Uploaded: processed/photo1_thumb.jpg Publishing SNS
notification... SUCCESS END RequestId: 3f2a-91cd Duration: 812 ms Billed Duration: 900 ms
```

The subscribed email/webhook is expected to receive a notification similar to:

```
Subject: Image Processed Successfully { "file": "photo1.jpg", "thumbnail": "photo1_thumb.jpg",
"status": "SUCCESS", "processedAt": "2026-08-12T10:15:32Z" }
```

For an invalid file upload, the function log is expected to show a validation failure and the notification service is expected to relay a corresponding failure alert instead of a success message, confirming the branching event logic works correctly.

5. Conclusion

This experiment demonstrated a complete serverless, event-driven pipeline built entirely from managed cloud services — object storage as both the trigger source and the sink, a FaaS component for stateless, on-demand compute, and a notification service for asynchronous alerting. No server was provisioned or managed at any point, and the architecture automatically scales with the number of concurrent uploads since each event independently invokes its own function instance.

The pattern illustrated here — storage-triggered functions feeding into pub/sub notification systems — is a foundational serverless design used widely for media processing, log processing, and IoT data ingestion pipelines. Its key benefits are cost efficiency (billing only for actual execution time), automatic scalability, and reduced operational overhead compared to a traditional server-based image-processing service.

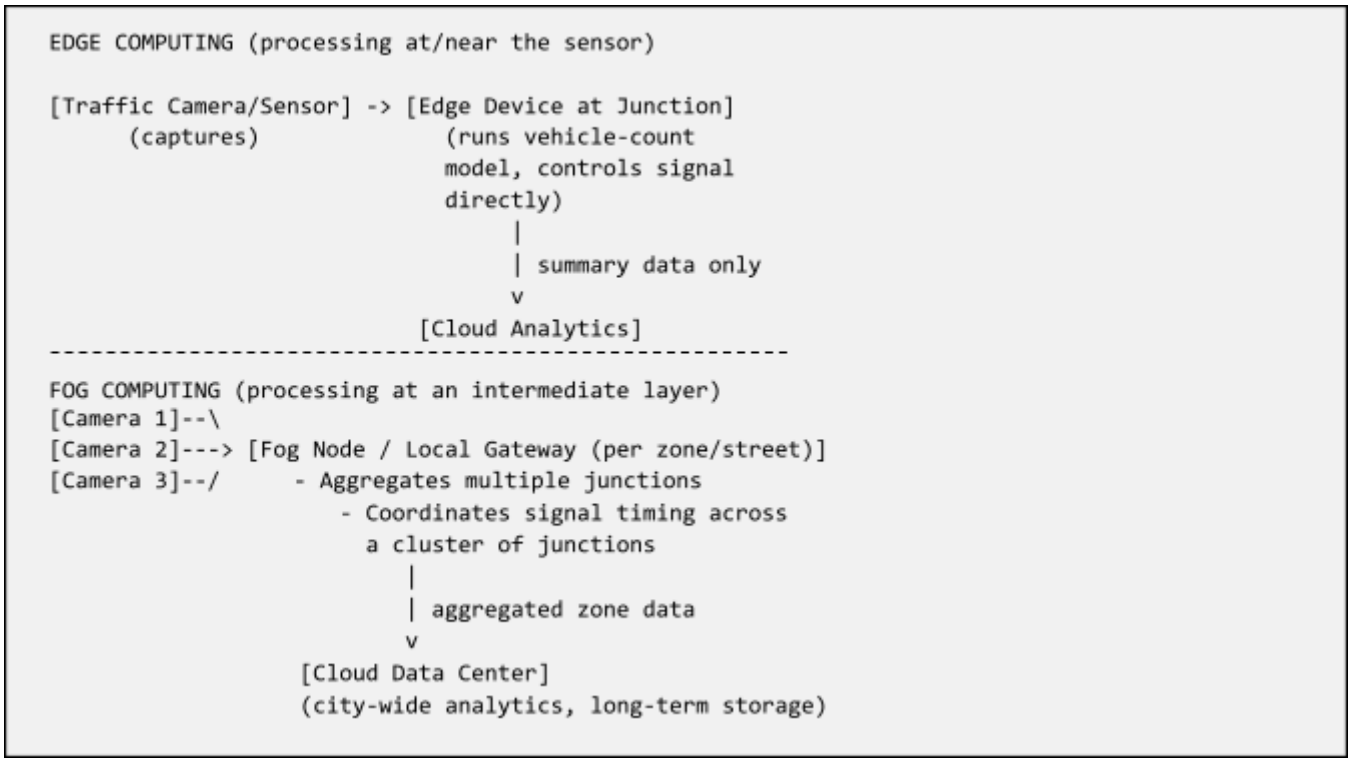
Experiment 4: Edge Computing vs Fog Computing for Smart Traffic Management

1. Objective

The objective of this experiment is to compare edge computing and fog computing architectures in the context of a smart traffic management system, and to evaluate their respective architectures, latency characteristics, and deployment locations. The system under study uses roadside sensors and cameras to detect traffic density and dynamically control signal timing.

The comparison aims to determine which computing paradigm — or combination of both — is best suited to meet the real-time responsiveness required for traffic signal control while still supporting city-wide analytics in the cloud.

2. Architecture Sketch



3. Procedure

24. Define the traffic management use case: cameras at each junction detect vehicle density and the system must adjust signal timing within milliseconds to seconds to avoid congestion.
25. Model the Edge Computing architecture: deploy a lightweight inference model directly on a compute unit attached to each junction's camera, so vehicle counting and signal decisions happen locally with no network round-trip.
26. Model the Fog Computing architecture: deploy an intermediate fog node/gateway serving a cluster of nearby junctions (e.g., a street or district), which aggregates data from multiple edge sensors and coordinates signal timing across the cluster (e.g., green-wave synchronisation).
27. Simulate/estimate the latency for each architecture: edge (sensor -> local decision, no network hop) versus fog (sensor -> fog node over LAN) versus a pure cloud model (sensor -> data center over WAN) as a baseline.

28. Tabulate deployment location, hardware footprint, coordination scope, and latency for each paradigm.
29. Evaluate which paradigm best serves (a) the sub-second signal control requirement, and (b) city-wide, multi-junction coordination and analytics.
30. Propose a hybrid design combining both, and describe the resulting data flow from edge to fog to cloud.

4. Expected Output

Aspect	Edge Computing	Fog Computing
Deployment Location	On/near the sensor (junction device)	Intermediate gateway (zone/street level)
Processing Scope	Single junction	Cluster of junctions
Typical Latency	~1-10 ms (local decision)	~10-50 ms (LAN hop to gateway)
Coordination Capability	None beyond local junction	Cross-junction signal synchronisation
Hardware Footprint	Small, embedded device per camera	Moderate, one gateway per zone
Best Suited For	Immediate local signal actuation	Zone-level traffic-flow optimisation

The recommended hybrid deployment is: edge devices perform immediate, junction-local signal control to guarantee sub-10 ms response for pedestrian/vehicle safety, while fog nodes aggregate data from clusters of junctions to synchronise green-wave timing along a corridor; both tiers forward summarised data to a central cloud analytics platform for city-wide traffic pattern analysis, historical reporting, and long-term optimisation.

5. Conclusion

The comparison shows that edge and fog computing address complementary needs rather than being mutually exclusive alternatives. Edge computing delivers the lowest possible latency by processing data at the point of generation, making it indispensable for time-critical, safety-related decisions such as immediate signal actuation. Fog computing, positioned one layer above the edge, trades a small amount of latency for the ability to coordinate and aggregate data across multiple sensors, enabling zone-level optimisations such as green-wave signal synchronisation that a single edge device cannot achieve alone.

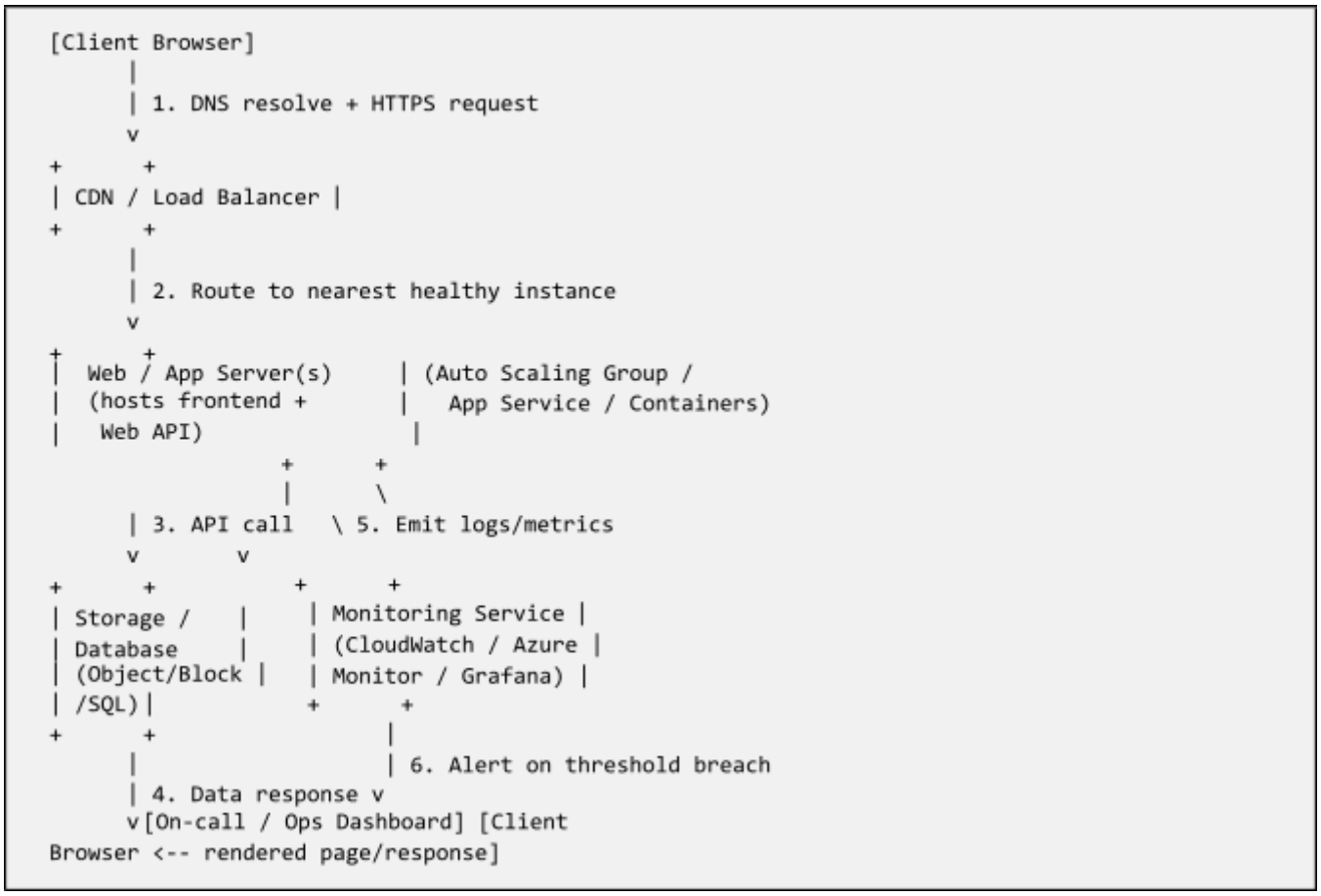
For a smart traffic management system, neither paradigm alone is sufficient: a hybrid edge-fog-cloud architecture provides the best balance — edge nodes for instantaneous local control, fog nodes for regional coordination, and the cloud for city-wide, long-term analytics and machine-learning model retraining, which is then periodically redeployed back down to the edge devices.

Experiment 5: Cloud Application Deployment Workflow

1. Objective

The objective of this experiment is to demonstrate a complete cloud application deployment workflow, tracing the path from client access through browser interaction, web API invocation, backend storage access, and continuous monitoring. The experiment consolidates concepts from client-server architecture, RESTful APIs, cloud storage, and observability into a single end-to-end deployment scenario.

2. Architecture Sketch



3. Procedure

31. Package the web application (frontend static assets + backend Web API) and deploy it to a cloud compute service (e.g., an Auto Scaling Group of EC2 instances, Azure App Service, or a container on ECS/AKS).
32. Place a load balancer / CDN in front of the deployed instances to distribute incoming client traffic and terminate HTTPS.
33. From a client browser, navigate to the application's public URL; observe DNS resolution and the resulting HTTPS request reaching the load balancer, which forwards it to a healthy backend instance.
34. Within the application, trigger a Web API call (e.g., clicking a 'Load Data' button) and trace the request from browser -> load balancer -> app server -> API handler.
35. Have the API handler read/write data from a cloud storage or database service, and return the response back through the same path to the browser, where it is rendered.

36. Enable a monitoring/observability service to collect application logs, request latency, error rates, and resource utilisation metrics from the deployed instances.
37. Configure an alert rule (e.g., error rate > 5% or CPU > 80%) so that the monitoring service notifies the operations team automatically when a threshold is breached.
38. Simulate load (e.g., using a load-testing tool) to observe auto-scaling behaviour and confirm that new instances are provisioned automatically and monitoring dashboards reflect the increased load in real time.

4. Expected Output

A successful end-to-end request is expected to complete with the following observable trail:

```
[Browser]        GET https://portal.example.com/dashboard          -> 200 OK (312 ms) [Load Balancer]
Routed to instance i-0abc123 (healthy) [Web API]   GET /api/dashboard/summary
-> 200 OK (85 ms) [Database] SELECT ... FROM traffic_data -> 42 ms [Monitoring] RequestCount
+1, AvgLatency 312ms, ErrorRate 0%
```

The monitoring dashboard is expected to display live graphs of request count, latency, and error rate, and to trigger a configured alert (e.g., via email or Slack webhook) if the simulated load pushes CPU utilisation or error rate above the defined threshold, confirming that the auto-scaling and alerting mechanisms function as designed.

5. Conclusion

This experiment traced a complete cloud application request lifecycle — from client browser through load balancing, application logic, storage access, and back to the client — while simultaneously capturing the operational visibility layer (monitoring and alerting) that keeps such a deployment reliable in production. It demonstrated how each architectural component (CDN/load balancer, auto-scaled compute, storage, and monitoring) plays a distinct role in ensuring the application is available, performant, and observable.

The exercise reinforces that a production-grade cloud deployment is not merely about hosting code on a server; it requires an integrated workflow of traffic distribution, elastic scaling, secure data access, and continuous monitoring with automated alerting. This end-to-end view equips a cloud practitioner to design deployment pipelines that remain resilient and responsive under varying load conditions, which is precisely the outcome demonstrated in this experiment.